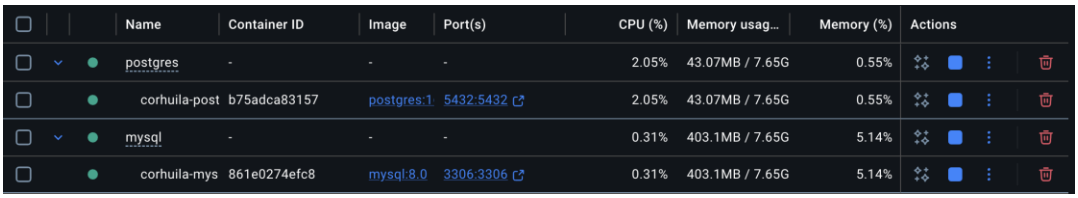
1. Verification Docker and Docker Compose.

```
❯ docker --version
Docker version 29.2.1, build a5c7197
❯ docker compose version
Docker Compose version v5.0.2
```

2. Setting up PostgreSQL in a container.

```
❯ docker compose up -d
[+] up 2/2
 ✔ Network postgres_default    Created
 ✔ Container corhuila-postgres Created
```

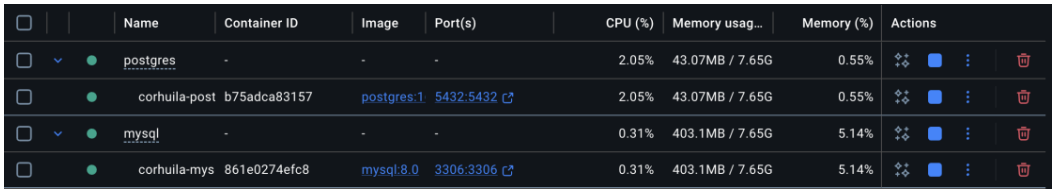


Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Actions
postgres	-	-	-	2.05%	43.07MB / 7.65G	0.55%	
corhuila-post	b75adca83157	postgres:1	5432:5432	2.05%	43.07MB / 7.65G	0.55%	
mysql	-	-	-	0.31%	403.1MB / 7.65G	5.14%	
corhuila-mys	861e0274efc8	mysql:8.0	3306:3306	0.31%	403.1MB / 7.65G	5.14%	

```
❯ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                    PORTS                                                NAMES
861e0274efc8   mysql:8.0            "docker-entrypoint.s…"   3 minutes ago    Up 3 minutes (healthy)    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp          corhuila-mysql
b75adca83157   postgres:16-alpine   "docker-entrypoint.s…"   4 minutes ago    Up 4 minutes (healthy)    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp          corhuila-postgres
```

3. Setting up MySQL in a container.

```
❯ docker compose up -d
[+] up 2/2
 ✔ Network mysql_default    Created
 ✔ Container corhuila-mysql Created
```



Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Actions
postgres	-	-	-	2.05%	43.07MB / 7.65G	0.55%	
corhuila-post	b75adca83157	postgres:1	5432:5432	2.05%	43.07MB / 7.65G	0.55%	
mysql	-	-	-	0.31%	403.1MB / 7.65G	5.14%	
corhuila-mys	861e0274efc8	mysql:8.0	3306:3306	0.31%	403.1MB / 7.65G	5.14%	

```
❯ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                    PORTS                                                NAMES
861e0274efc8   mysql:8.0            "docker-entrypoint.s…"   3 minutes ago    Up 3 minutes (healthy)    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp          corhuila-mysql
b75adca83157   postgres:16-alpine   "docker-entrypoint.s…"   4 minutes ago    Up 4 minutes (healthy)    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp          corhuila-postgres
```

4. Create the database in both engines.

a. MySQL

```
CREATE DATABASE laboratorio_docker;
USE laboratorio_docker;
```

b. PostgreSQL

```
CREATE DATABASE laboratorio_docker;
\c laboratorio_docker;
```

5. Create the student entity.

a. MySQL

```
CREATE TABLE estudiante (
    id INT AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(20) NOT NULL UNIQUE,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(120) NOT NULL UNIQUE,
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

b. PostgreSQL

```
CREATE TABLE estudiante (
    id SERIAL PRIMARY KEY,
    codigo VARCHAR(20) NOT NULL UNIQUE,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(120) NOT NULL UNIQUE,
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

6. Insert and query a record.

a. MySQL

```
INSERT INTO estudiante (codigo, nombre, correo)
VALUES ('2026001', 'Ana Torres', 'ana.torres@corhuila.edu.co');

SELECT * FROM estudiante;
```

b. PostgreSQL

```
INSERT INTO estudiante (codigo, nombre, correo)
VALUES ('2026001', 'Ana Torres', 'ana.torres@corhuila.edu.co');

SELECT * FROM estudiante;
```

7. To run a .sql script, the following command was used as STDIN:

a. MySQL

```
❯ docker exec -i corhuila-mysql mysql -u root -pmysql123 mysql < sql/bd.sql
```

b. PostgreSQL

```
❯ docker exec -i corhuila-postgres psql -U postgres -d postgres < sql/bd.sql
```

8. Run docker exec -it for Allocate a pseudo-TTY:

a. MySQL

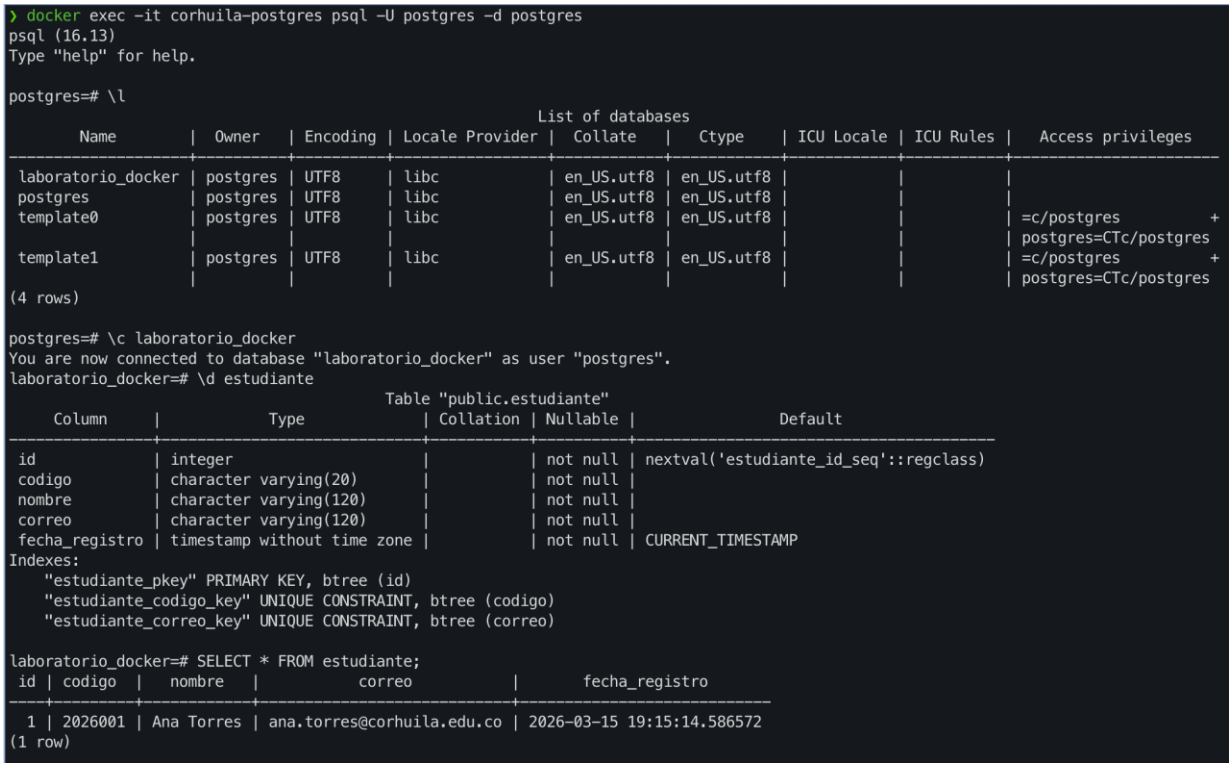
```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| laboratorio_docker |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> USE laboratorio_docker
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> DESCRIBE estudiante;
+----------------+-------------+------+-----+-------------------+-------------------+
| Field          | Type        | Null | Key | Default           | Extra             |
+----------------+-------------+------+-----+-------------------+-------------------+
| id             | int         | NO   | PRI | NULL              | auto_increment    |
| codigo         | varchar(20) | NO   | UNI | NULL              |                   |
| nombre         | varchar(120)| NO   |     | NULL              |                   |
| correo         | varchar(120)| NO   | UNI | NULL              |                   |
| fecha_registro | timestamp   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+-------------+------+-----+-------------------+-------------------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM estudiante;
+----+---------+------------+---------------------------+---------------------+
| id | codigo  | nombre     | correo                    | fecha_registro      |
+----+---------+------------+---------------------------+---------------------+
|  1 | 2026001 | Ana Torres | ana.torres@corhuila.edu.co | 2026-03-15 19:22:29 |
+----+---------+------------+---------------------------+---------------------+
1 row in set (0.00 sec)
```

b. PostgreSQL



```
❯ docker exec -it corhuila-postgres psql -U postgres -d postgres
psql (16.13)
Type "help" for help.

postgres=# \l
                                                    List of databases
       Name        |  Owner   | Encoding | Locale Provider |  Collate   |   Ctype    | ICU Locale | ICU Rules |   Access privileges
-------------------+----------+----------+-----------------+------------+------------+------------+-----------+-----------------------
 laboratorio_docker | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           |
 postgres           | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           |
 template0          | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           | =c/postgres          +
                    |          |          |                 |            |            |            |           | postgres=CTc/postgres
 template1          | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           | =c/postgres          +
                    |          |          |                 |            |            |            |           | postgres=CTc/postgres
(4 rows)

postgres=# \c laboratorio_docker
You are now connected to database "laboratorio_docker" as user "postgres".
laboratorio_docker=# \d estudiante
                                      Table "public.estudiante"
     Column     |            Type             | Collation | Nullable |                 Default
----------------+-----------------------------+-----------+----------+----------------------------------------
 id             | integer                     |           | not null | nextval('estudiante_id_seq'::regclass)
 codigo         | character varying(20)       |           | not null |
 nombre         | character varying(120)      |           | not null |
 correo         | character varying(120)      |           | not null |
 fecha_registro | timestamp without time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "estudiante_pkey" PRIMARY KEY, btree (id)
    "estudiante_codigo_key" UNIQUE CONSTRAINT, btree (codigo)
    "estudiante_correo_key" UNIQUE CONSTRAINT, btree (correo)

laboratorio_docker=# SELECT * FROM estudiante;
 id | codigo  |   nombre    |          correo           |       fecha_registro
----+---------+-------------+---------------------------+----------------------------
  1 | 2026001 | Ana Torres | ana.torres@corhuila.edu.co | 2026-03-15 19:15:14.586572
(1 row)
```

Brief reflection comparing PostgreSQL and MySQL

PostgreSQL and MySQL are two of the most widely used relational database management systems in software development. Both provide high performance, reliability, and strong support for modern applications, especially in web and backend environments. However, they differ in their design philosophy and strengths. **MySQL** is known for its simplicity, ease of configuration, and widespread use in traditional web applications, making it very popular in environments such as the LAMP stack. On the other hand, **PostgreSQL** is recognized for its robustness, strict adherence to SQL standards, and its ability to handle complex queries, advanced extensions, and large volumes of data.

In conclusion, while MySQL is often preferred for its ease of use and efficiency in common web applications, PostgreSQL is valued for its power and flexibility in systems that require more complex data management. The choice between them ultimately depends on the specific needs and complexity of the project.